EXPERIMENT 5: INFORMATION RETRIEVAL USING spaCy, TF-IDF AND COSINE SIMILARITY

AIM:

To implement an information retrieval system using the 20 Newsgroups dataset by preprocessing text with spaCy, extracting stop words, examining part-of-speech tags, converting documents into TF-IDF vectors, and ranking documents relevant to the query “NASA space launch” using cosine similarity.

ALGORITHM:

1. Import spaCy, Pandas, fetch_20newsgroups, TfidfVectorizer, and cosine_similarity.

2. Load the English spaCy language model and fetch the 20 Newsgroups dataset, excluding headers, footers, and quoted text.

3. Select the first 500 documents and store them in a Pandas DataFrame.

4. Define a preprocessing function that converts text to lowercase, tokenizes it with spaCy, removes stop words and non-alphabetic tokens, and uses token lemmas.

5. Apply the preprocessing function to create a cleaned-text column.

6. Extract and display stop words from sample documents.

7. Display part-of-speech tags for tokens in the first three documents.

8. Use TfidfVectorizer to convert the cleaned documents into TF-IDF vectors.

9. Preprocess the query “NASA space launch” and transform it into a TF-IDF vector.

10. Calculate cosine similarity between the query vector and the document vectors.

11. Rank the similarity scores and display the top five relevant documents.

```python
# EXPERIMENT-3c Muthuraj(241801176)

# Step 1: Load Dataset

import spacy
import pandas as pd
import string

from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

nlp = spacy.load("en_core_web_sm")

newsgroups = fetch_20newsgroups(
    subset='all',
        remove=('headers', 'footers', 'quotes')
        )

documents = newsgroups.data[:500]
```

```python
# EXPERIMENT-3c Muthuraj(241801176)

# Step 2: Preprocessing with spaCy

def preprocess_spacy(text):
    doc = nlp(text.lower())
    tokens = [
        token.lemma_
        for token in doc
         if token.is_alpha and not token.is_stop
    ]
    return ' '.join(tokens)
```

```python
# EXPERIMENT-3C Muthuraj(241801176)

# Step 3: Apply preprocessing

df = pd.DataFrame({'Text': documents})
df['clean_text'] = df['Text'].apply(preprocess_spacy)
```

```python
# EXPERIMENT-3CMuthuraj(241801176)

# Step 4: Function to extract stop words as a space-separated string

def extract_stop_words(text):
    doc = nlp(text.lower())
    stop_words_in_text = [
        token.lemma_
        for token in doc
        if token.is_stop and token.is_alpha
    ]
    return ' '.join(stop_words_in_text)

df['stop_words'] = df['Text'].apply(extract_stop_words)
df[['stop_words']].head()
```

	stop_words
0	I be some of be about the of any of about the ...
1	my be in the for a that with do anyone have on...
2	you what you about that be the will be after s...
3	it the do the not the the can do from any of t...
4	I have an which I can not with my my be that I...

```python
# EXPERIMENT-3C Muthuraj(241801176)

# Step 5: POS Tagging example (first few docs)

for i, doc in enumerate(df['Text'][:3]):
    print(f"\nDocument {i+1} POS Tags:")
    spacy_doc = nlp(doc)
    print([(token.text, token.pos_) for token in spacy_doc[:10]])
```

```
Document 1 POS Tags:
[('\n\n', 'SPACE'), ('I', 'PRON'), ('am', 'AUX'), ('sure', 'ADJ'), ('some', 'DET'), ('bashers', 'NOUN'), ('of', 'ADP'), ('Pens', 'PROPN'), ('fans', 'NOUN'), ('are', 'AUX')]

Document 2 POS Tags:
[('My', 'PRON'), ('brother', 'NOUN'), ('is', 'AUX'), ('in', 'ADP'), ('the', 'DET'), ('market', 'NOUN'), ('for', 'ADP'), ('a', 'DET'), ('high', 'ADJ'), ('-', 'PUNCT')]

Document 3 POS Tags:
[('\n\n\n\n\t', 'SPACE'), ('Finally', 'ADV'), ('you', 'PRON'), ('said', 'VERB'), ('what', 'PRON'), ('you', 'PRON'), ('dream', 'VERB'), ('about', 'ADP'), ('.', 'PUNCT'), ('Mediterrane
```

```python
# EXPERIMENT-3C Muthuraj(241801176)
# Step 6: Vectorization using TF-IDF

vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(df['clean_text'])
```

```
# EXPERIMENT-3C Muthuraj(241801176)
# Step 7: Query

query = "NASA space launch"
query_clean = preprocess_spacy(query)
query_vec = vectorizer.transform([query_clean])
```

```
# EXPERIMENT-3C Muthuraj(241801176)

# Step 8: Similarity & Ranking

similarities = cosine_similarity(
    query_vec,
    tfidf_matrix
).flatten()

top_indices = similarities.argsort()[-5:][::-1]
```

```
# EXPERIMENT-3C Muthuraj(241801176)

# Step 9: Display top 5 results (just the text)

print("\n🔍 Top 5 Relevant Documents:")

for i in top_indices:
    print(df.loc[i, 'Text'][:300])
    print()
```

```
🔍 Top 5 Relevant Documents:
I am posting this for a friend without internet access. Please inquire
to the phone number and address listed.
--------------------------------------------------------------------

"Space: Teaching's Newest Frontier"
Sponsored by the Planetary Studies Foundation

The Planetary Studies Foundation is

: >Why not build a inflatable space dock.

: If you're doing large-scale satellite servicing, being able to do it in
: a pressurized hangar makes considerable sense.  The question is whether
: anyone is going to be doing large-scale satellite servicing in the near
: future, to the point of justifyin

"Space Station Redesign Leader Says Cost Goal May Be
Impossible"

Today (4/6) the Washington Post ran an article with the
```

RESULT:

The information retrieval program processed the 20 Newsgroups text collection using spaCy, extracted stop words, and displayed part-of-speech tags for sample documents. The documents were converted into TF-IDF vectors, and cosine similarity was used to rank documents for the query “NASA space launch”. The top five relevant documents were displayed.